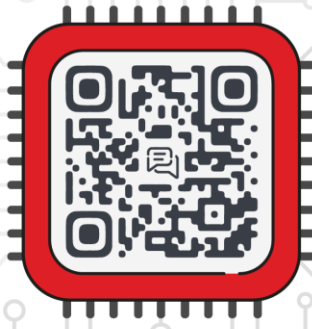




نظري

1

24

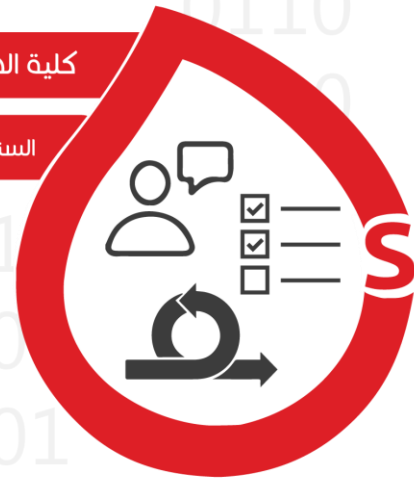
3600 sp

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Review

د. سعيد أبو طراب



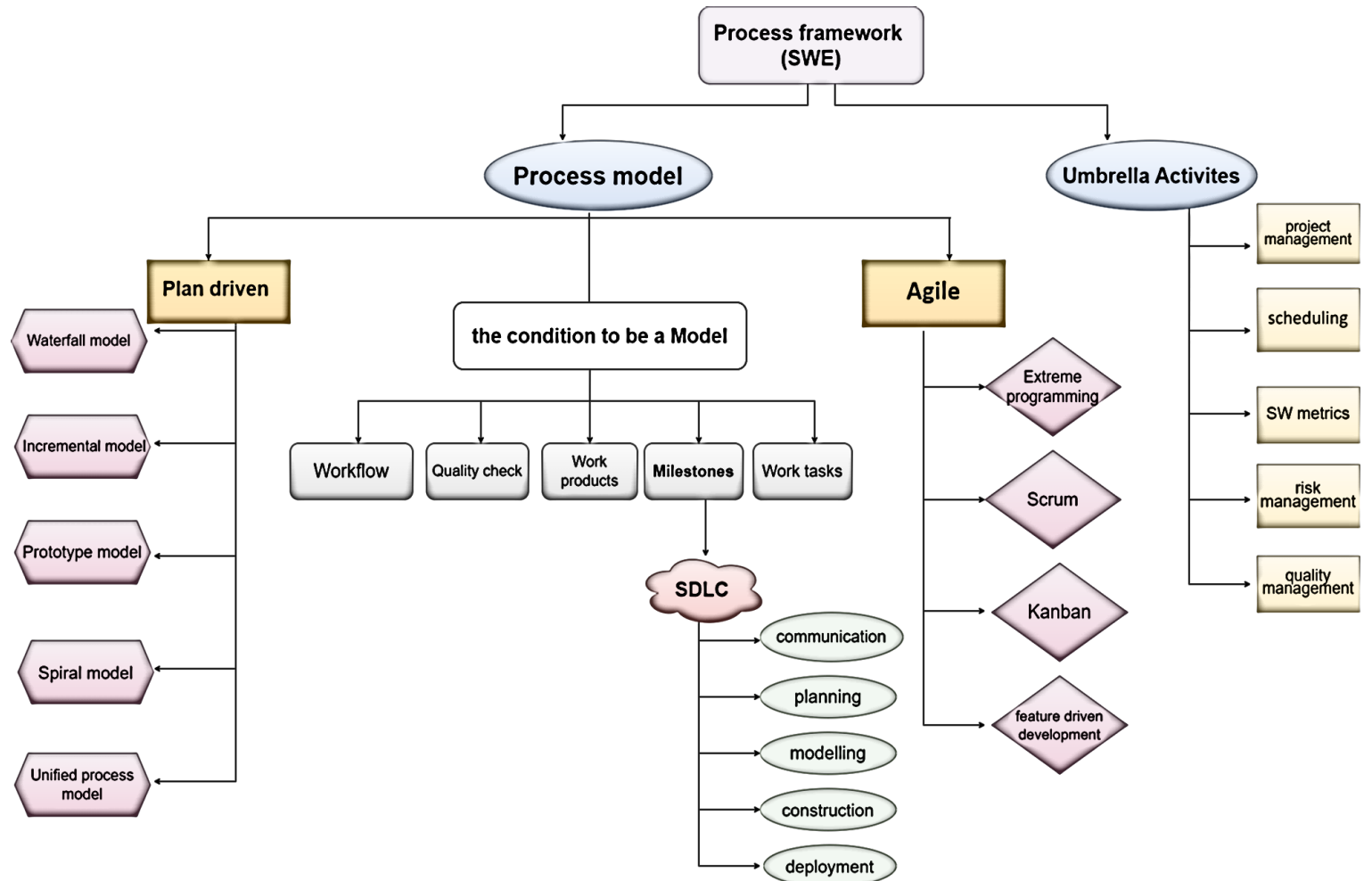
RB Informatics; 01/04/2026

هندسة برمجيات ٢

السلام عليكم يا أصدقاء، في هذه المحاضرة سنستعرض بشكل مختصر وشامل أهم المفاهيم التي تناولناها في مادة هندسة البرمجيات 1 خلال الفصل الماضي، حيث سنعيد ترتيب الأفكار الأساسية ونلخص المراحل والخطوات التي تمر بها عملية تطوير البرمجيات.

يهدف هذا الملخص إلى ترسيخ المعلومات السابقة وتسهيل الانتقال إلى المفاهيم الجديدة في مادة هندسة البرمجيات 2، من خلال تقديم صورة واضحة ومتكاملة عن أساسيات المادة.

الخريطة المفاهيمية للمادة



❖ مفهوم هندسة البرمجيات:

هي تخصص هندسي يهتم بجميع جوانب إنتاج البرمجيات، من مراحل تحديد النظام المبكرة وحتى صيانة النظام بعد أن يبدأ العمل به، وإدارة المشاريع وتطوير الأدوات والأساليب وما إلى ذلك للحصول على برمجيات اقتصادية موثوقة وتعمل بكفاءة على الأجهزة الفعلية.

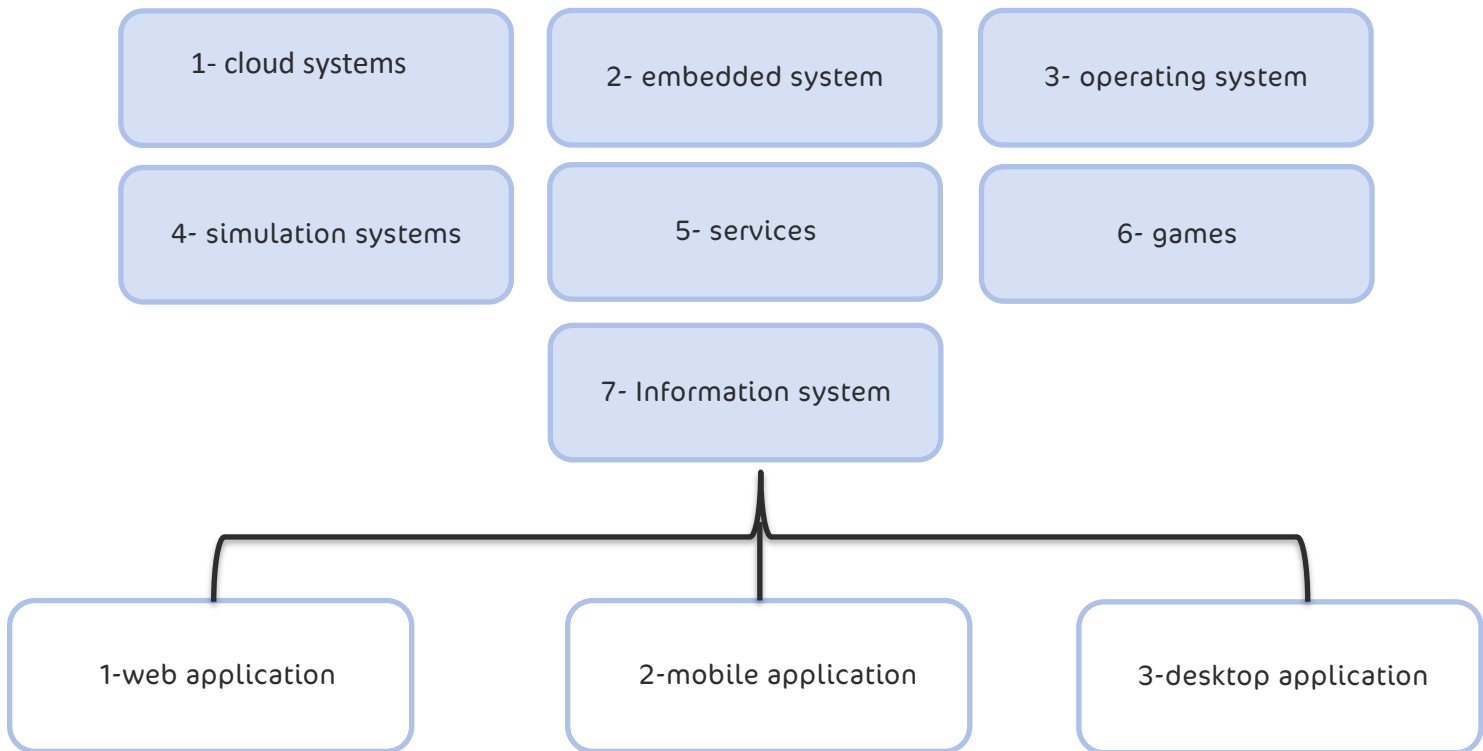
فالهندسة هي إجرائية process هدفها حل المشكلات والتي تعتمد على معيارين أساسيين وهما:

كفاءة استخدام الموارد

الجودة

فيكون الحل أمثلياً عندما ننتج منتجاً **بأعلى** جودة ممكنة وباستهلاك **أقل** قدر ممكن من الموارد ضمن الموارد المتاحة، أي جودة مرتفعة وتكلفة منخفضة وزمن مقبول.

❖ أنواع البرمجيات من الناحية التقنية:



❖ أنواع البرمجيات من حيث الوصول الى الزبون:

1. المنتج الأفقي (horizontal product)

صاحب الشركة البرمجية هو من يحدد المتطلبات للتطبيق البرمجي وفقاً لحاجة السوق.

1. المنتج العمودي (vertical product)

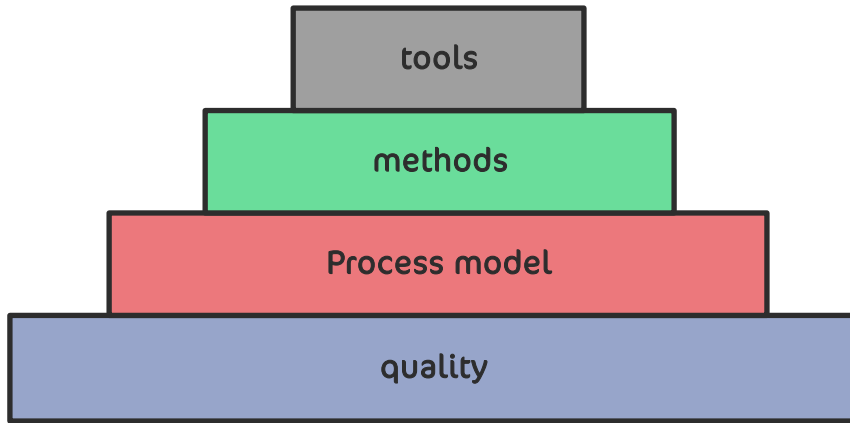
هو منتج يتم تحديد متطلباته من قبل الزبون ويعد أقل خطورة من الـ horizontal لأنه يركز على عميل محدد.

أخلاقيات المهنة:

يجب على مهندسي البرمجيات الالتزام بـ:

1. **السرية:** عدم تسريب أية معلومات واحترام سرية أصحاب العمل.
2. **حسن استخدام الحاسب:** عدم استخدام مهاراتهم التقنية لإساءة استخدام حواسيب الآخرين.
3. **حقوق الملكية الفكرية:** أن يكونوا على دراية بالقوانين المحلية التي تحكم استخدام الملكية الفكرية.
4. **الكفاءة:** عدم التظاهر بامتلاكهم مستوى أعلى من كفاءتهم، وعدم قبول الأعمال التي تفوق مستوى كفاءتهم.

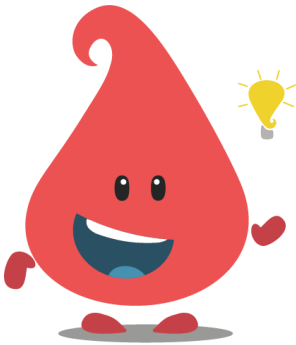
التمثيل الهرمي لهندسة البرمجيات:



Process Model

تعرف الـ **Process Model** على أنها مجموعة مُنظمة من الأنشطة المطلوبة لتطوير نظام برمجي وتمثل نموذجاً مهيأ لإعادة الاستخدام.

الشروط الواجب توافرها في أي Process حتى تصبح Model:



- Work tasks ✓
- Milestones ✓
- Work products ✓
- Quality check ✓
- Activity flow أو Workflow ✓

(SDLC) Software Development Life Cycle

هو مصطلح يوصف المراحل milestones التي يمكن المرور بها للوصول الى منتج برمجي ويعبر عن دورة حياة تطوير المنتج، ويقصد بدورة الحياة المسيرة الكاملة للمشروع بدءاً من لحظة ولادة المشروع "طلب العميل له" وحتى لحظة الممات "تسليم المشروع وانتهاء فترة (الكفالة)"

communication → planning → modelling → construction → deployment

Process Models

1. Plan driven (ordinary process model) النماذج الإجرائية التقليدية:

وهي الإجرائيات ذات التخطيط المسبق، يعتمد هذا الأسلوب على التخطيط لكل شيء قبل البدء فيه، أي عدم التحرك حتى تتمكن من معرفة جميع الخطوات التي سنقوم بها، ومن سيقوم بها، وكم نحتاج من الوقت لتنفيذها،

Waterfall model النموذج الشلالي

ينفذ مراحل الـ SDLC (milestones) بشكل خطي وباتجاه واحد غير قابل للعودة، أي أنه يقوم بتنفيذ كل مرحلة من مراحل المشروع وإنهاءها بشكل كامل ومن ثم ينتقل إلى المرحلة التي تليها. يتميز بالوضوح ولا يقبل تغيير المتطلبات يستخدم لتطوير المشاريع الثابتة ذات المتطلبات الواضحة والتي تحوي على مرجعية معينة، يعمل للمشاريع الصغيرة.

السلبيات:

- صعوبة الاستجابة لمتطلبات الزبون المتغيرة، وعدم القدرة على التواصل معه مما قد يسبب عدم رضائه عن المنتج النهائي
- إنتاج مشاريع ثابتة غير قابلة للتعديل.
- لا يمكن الانتقال من مرحلة إلى أخرى دون الانتهاء بشكل كامل من المرحلة الحالية، مما يسبب هدر في الموارد وعدم إمكانية التراجع أو التعديل

الإيجابيات:

- صورة واضحة مع هيكلية قوية للمشروع لأنه يستعرض العناصر الأساسية مما يجعله سريع.
- تدفق خطي بين مراحل التطوير (تسلسلي)
- تقييم المشروع وتسليمه في نهاية عملية الكود والاختبار.
- منظم للغاية، يعتمد على التخطيط وشديد الانضباط

incremental model النموذج التزايدي

يقوم بتقسيم متطلبات المشروع العام إلى أجزاء (مشاريع جزئية) كل جزء يمثل دورة (increment) ويقدم جزء من الوظائف المطلوبة (جزء من المتطلبات الكلية)، وبعد إنهاء جميع الـ increments يتم تنفيذ عملية دمج "integration" لكل هذه الأجزاء معا وبناء مشروع واحد متكامل لتسليمه.

السلبيات:

- تميل بنية النظام إلى التدهور عند إضافة متطلبات جديدة
- يستغرق وقتاً أكبر وليس واضحاً كالشلالي.
- تقسيم المتطلبات يحتاج إلى خبرة كبيرة، وصعوبة الدمج "integration"

الإيجابيات:

- تخفيض تكلفة إنجاز المتطلبات المتغيرة للزبون.
- التواصل الدائم مع الزبون وأخذ feedback منه على العمل المنجز.
- قدرة أكبر على تسليم سريع للوظائف الأساسية المفيدة واستفادة الزبون منها.
- العمل بشكل متواز.

Prototyping model

هو نسخة أولية من النظام يُستخدم بشكل أساسي لجمع المتطلبات وتحديدتها بدقة مع الزبون، بحيث نقوم بتصميم واجهات أولية UI للتطبيق مع إهمال الوظيفة وعرضها على الزبون، ومن ثم أخذ feedback عنها وتعديلها وفقاً لمتطلباته، نقوم بتكرار هذه العملية إلى أن يتم الاتفاق بشكل نهائي على واجهات التطبيق وما سيتم تقديمه من خدمات ووظائف وميزات ضمنه.

يتم استخدامه مع النماذج الإجرائية الأخرى التي ذكرناها.

الإيجابيات:

- وسيلة ممتازة لمعرفة متطلبات الزبون
- تحسين قابلية استخدام النظام
- تحسين جودة التصميم وقابلية الصيانة.

السلبات:

- هدر كبير للوقت
- عادةً ما تكون النماذج الأولية غير موثقة
- من المحتمل ألا يلبي معايير الجودة التنظيمية

Spiral model النموذج الحلزوني

هو نموذج تسلسلي تزايدى تكراري، يتم فيه دمج ميزات النماذج الإجرائية الأخرى، ويعد أكثر نماذج الـ SDLC شهرة.

آلية عمله:

يستخدم نهجاً تكرارياً ويتألف من عدة دورات iterations وتتألف كل دورة من مراحل تطوير البرنامج التقليدية جميعها.

الإيجابيات:

- يحوي كمية كبيرة من تحليل المخاطر.
- مناسب للمشاريع الكبيرة والمهمة.
- يمكن إضافة وظائف إضافية في وقت لاحق.
- مراقبة قوية للتوثيق.

السلبات:

- نجاح المشروع يعتمد بشكل كبير على مرحلة تحليل المخاطر.
- نموذج معقد يحتاج لفريق عمل كبير نسبياً
- يحتاج وقت كثير وهو مكلف للغاية

الفرق الحقيقي بين النموذج التزايدى والنموذج الحلزوني:

Spiral: يعمل على التسلسل فقط ولا يمكن البدء بمرحلة جديدة إلا بعد الانتهاء من جميع المراحل السابقة، وبكل دورة نطبق مراحل التطوير جميعها.

Incremental: يمكننا القيام بتطوير الـ increments على التسلسل أو على التوازي لتسريع العملية، وذلك بأن يقوم أكثر من فريق بتطوير كل increment مستقلة على حدى.

Unified Process Model

هو نموذج اجرائي عام وموحد، تسلسلي تزايدى وتكراري يعتمد على تحليل المخاطر، ويعدّ دليل للاستخدام الفعّال للغة النمذجة الموحدة UML، ولا يعتمد مبدأ الـ SDLC ولا يوجد فيه فواصل حدية بين الأطوار.

الإيجابيات:

- إبقاء الـ Stakeholders على اطلاع بكامل التطورات والمحافظة على استقبال feedback منهم.
- التعاون بين الفرق العاملة في كافة مراحل تنفيذ المشروع من خلال الحفاظ على التواصل المفتوح بينهم.
- تسليم نسخة جاهزة من النظام في وقت مبكر وبشكل منتظم، واكتشاف المشكلات في وقت مبكر من التطوير.
- التركيز على الجودة، وكفاءة استخدام الموارد
- تحسين السيطرة وإدارة المخاطر.

السلبيات:

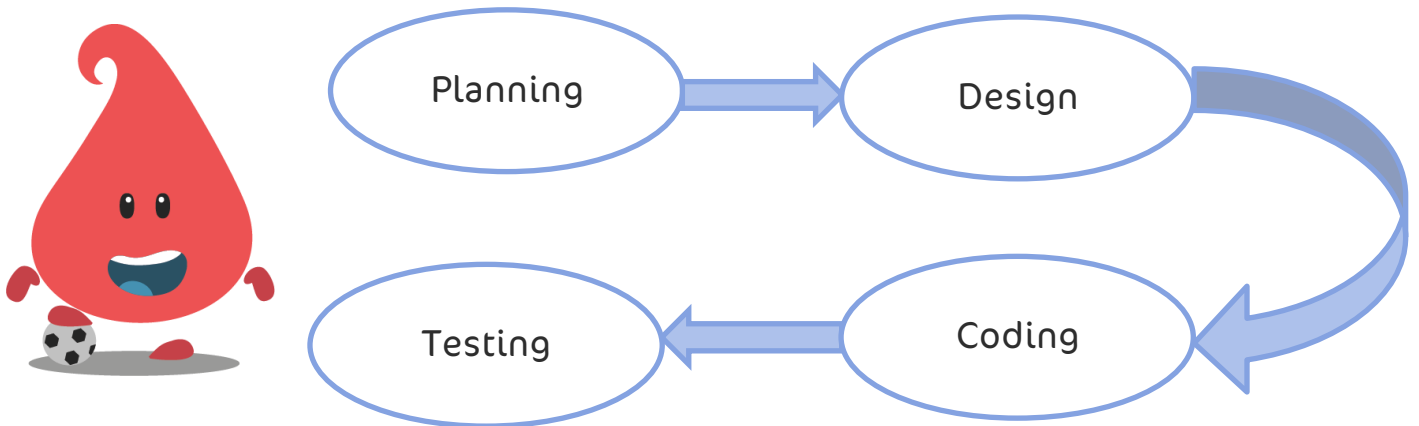
- قد تكون العملية معقدة جداً للتنفيذ.
- يمكن أن يخرج التطوير عن السيطرة ونتجه باتجاه العشوائية بالعمل.
- تعدّ عملية ثقيلة ومكلفة جداً.
- نحتاج إلى خبير لاعتماد هذه العملية بالكامل.

2. Agile development النماذج الإجرائية الرشيقة:

يكون التخطيط فيها قصير وغير مكلف أي عبارة عن short cycle، تتميز بسهولة تعديل العملية لتعكس متطلبات العملاء المتغيرة، فيمكن تطوير النظام بواسطة فريق صغير متواجد في نفس الموقع ويمكنه التواصل بشكل غير رسمي.

Extreme Programming (XP)

سمّيت بالبرمجة الحدية لأنه يتم فيها الاعتماد على coding لأقصى درجة ممكنة ويتم صرف الوقت الأكبر في هذه الإجرائية على هذه المرحلة، وتعتبر process لأن لدينا عدة milestones يتم القيام بها وهي:



والمراحل الأطول هنا هي الـ Coding والـ Testing.

Scrum Model

هي اجرائية Process مرنة تركّز على إدارة التطوير التكراري، تعد الأكثر انتشاراً وتعتبر Semi Structure أي تقوم بتحديد تعريفات وقواعد محددة يجب الالتزام بها، نشأت لضبط العمل فهي تقوم على متابعة العمل وتوصيف ماذا يحصل ومن المسؤول عنه بدقة، لتلافي الأخطاء الناتجة عن تهاون الأشخاص في تأدية مهامهم.

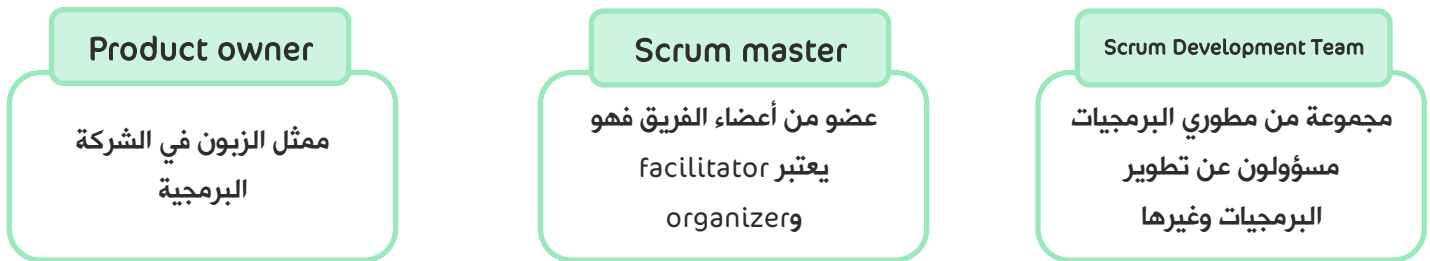
يوجد ثلاث مراحل في Scrum:

المرحلة الأولى: مرحلة التخطيط المبدئي

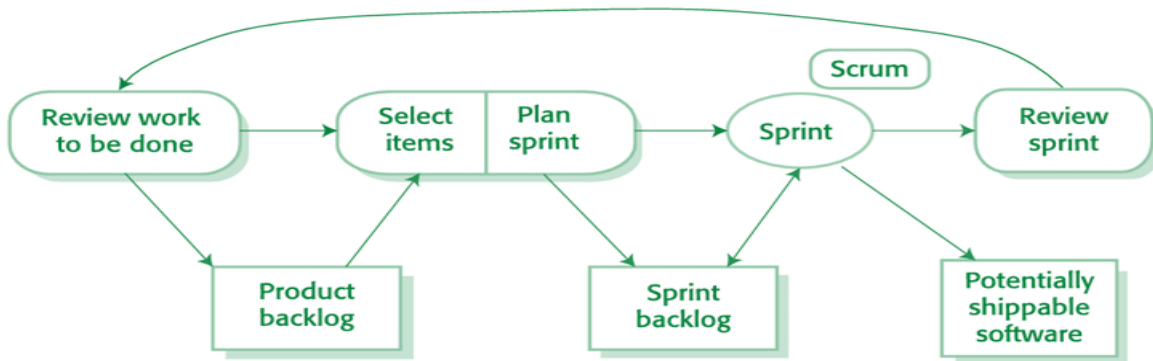
المرحلة الثانية: سلسلة دورات sprint cycles، حيث تطور كل دورة جزءاً متزايداً increment من النظام.

المرحلة الثالثة: مرحلة إغلاق المشروع، تختتم المشروع وتكمل الوثائق documentation المطلوبة.

أدوار Scrum:



Scrum Process



Kanban

- يعتمد kanban على طريقة تطوير unstructured أي غير مقيدة بمنهجية معينة وانما يعتمد على مبدأ تدفق المهام بحيث نقسم المشروع الى مجموعة من الـ tasks ونحدد أولوية كل مهمة ومن ثم نقوم بإضافتها الى جدول زمني على هيئة بطاقات بدون تحديد زمن لإنجاز المهمة بحيث نتمكن من رؤية حالة المهمة وتتبع التقدم ومعالجة أي عقبات أو عراقيل.
- تستند Kanban على فكرة التسليم المستمر، لا يوجد تسليم واحد ونهائي في نهاية العملية، وإنما سلسلة من التكرارات التي تظهر التقدم "Iterative".
- لدى منهجية Kanban بطاقات تمثل كل مهمة من مهام العمل التي يجب القيام بها في المشروع.

SDLC (software development life cycle)

1- Communication:

الهدف الأساسي من هذه المرحلة هو الحصول على المتطلبات ويعد فهم وتحديد المتطلبات أمراً هاماً جداً ومن أصعب المهام بسبب صعوبة التواصل بين الأطراف المعنية، ويكون أطراف التواصل:

1. Service provider (مقدم الخدمة): يمثل الشركة البرمجية Software company.
2. Service Seeker (طالب الخدمة): يمثل الشخص الذي يطلب المنتج عادة يسمى Customer ، وقد يكون (شخص / مجموعة أشخاص / شركة (صغيرة - متوسطة - كبيرة) / حكومة).

أنواع البنزس (Business Types):

■ B2G (Business to Government)

وهو القيام بتطوير برمجيات لصالح القطاع الحكومي.

■ B2B (Business to Business)

أي عندما نقوم بتقديم software لشركة محددة.

■ B2C (Business to Consumer/Customer)

أي عندما تقرر الشركة البرمجية ذاتها الخروج بفكرة تطبيق جديد نظراً لاحتياج لاحظته بالسوق.

Requirements (المتطلبات):

المتطلبات هي توصيف دقيق للخدمات التي يقدمها التطبيق والقيود التي يجب أن يعمل النظام وفقها، والخصائص التي يجب أن يتمتع بها، والتي تحدّد أثناء عملية هندسة المتطلبات.

المتطلبات = الخدمات + القيود + الخصائص

- الخدمات (Services): هي الميزات features التي سيقوم التطبيق بتقديمها.
- القيود (Constraints): هي القيود المفروضة على النظام مثل: التكلفة المرصودة، الوقت، الأدوات المتاحة، الخبرات، قوانين الدولة، قواعد الشركات، مثلاً هذه الشركة تقوم بالتطوير باستخدام framework معين.
- الخصائص (Properties): هي صفات الجودة للتطبيق مثل:

Performance (الأداء)

Scalability
(قابلية التوسع)

Usability
(سهولة الاستخدام)

Security (الأمان)

أنواع المتطلبات (Requirements Types)

يمكن أن تكون المتطلبات على مستويين مختلفين حسب الجهة المستهدفة:

■ متطلبات المستخدم (User Requirements)

وهي عبارات مكتوبة بلغة طبيعية بسيطة مع إمكانية إضافة مخططات توضح الخدمات التي يقدمها النظام والقيود التشغيلية الخاصة به.

لمن تكتب؟ تكتب خصيصاً لتكون مفهومة بالنسبة للعملاء أو المستخدمين الذين ليسوا تقنيين.

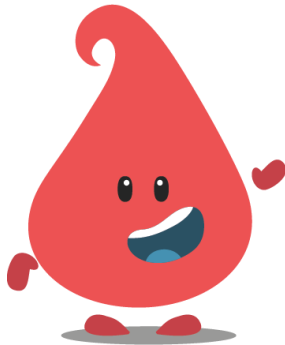
■ متطلبات النظام (System Requirements)

وثيقة منظمة ومفصلة تحتوي على أوصاف دقيقة لوظائف النظام وخدماته والقيود التشغيلية، تحدد ما يجب تنفيذه، لذا قد تكون جزءاً من عقد بين العميل والمقاول.

لمن تكتب؟ تكتب بشكل تقني للمطورين والمهندسين.

Requirement engineering هندسة المتطلبات

هي **process** هدفها الحصول على متطلبات ذات جودة عالية ضمن إطار زمني ومالي محدد، تتألف من عدة مراحل milestones وهي:



- Inception
- Elicitation
- Specification or Documentation
- Validation
- Management

المرحلة الأولى: Inception (التحضير)

يتم في هذه المرحلة "اكتشاف" المشروع وتحضير مكوناته، فيتم تحديد:

1. Scope (نطاق المشروع):

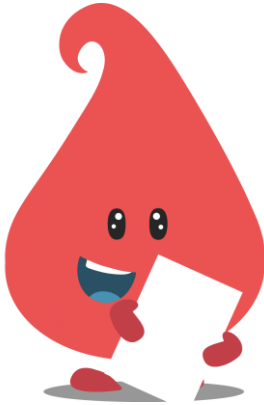
أي نحدد ما هي الميزات الداخلة ضمن المشروع وما التي هي خارجه (ما هو in وما هو out) بطريقة عامة ومجردة فنحدد:

- الهدف من النظام
- المشكلة الأساسية التي سنقوم بحلها
- نوع الـ business
- القطاع الذي سيخدمه المشروع Business domain بشكل خاص لا عام
- الجهة المستفيدة أو المستهدفة من النظام (target group).

2. Feasibility Study (دراسة الجدوى):

3. Stakeholders (أصحاب المصلحة):

4. Requirement gathering methods (طرق جمع المتطلبات):



- Domain search
- Brainstorming (العصف الذهني)
- Prototyping (النموذج الأولي)
- Sketching (الرسوم التخطيطية)
- Sampling (العينة)
- Persona (الشخصية)
- Interview (المقابلة)
- Survey (الاستبيان)
- Observation (المراقبة)

JAD: Join Application Development جمع ال Stakeholders في مكان واحد وتقسيمهم

إلى 3 مجموعات Users، Customers، Developers

المرحلة الثانية: Elicitation (الاستنباط):

يوجد عدة مراحل يجب تطبيقها:

1. Requirements discovery (اكتشاف المتطلبات)

2. Requirements classification and organization (تصنيف و تنظيم المتطلبات)

3. Requirements prioritization and negotiation (تحديد أولويات المتطلبات و التفاوض): حيث

تتم بناءً على عاملين أساسيين وهما ROI البدء بالمتطلبات التي تحقق للعميل قيمة و ربح أكثر و Dependency حسب اعتماد المتطلبات على بعضها البعض

4. Requirements specification (تحديد مواصفات المتطلبات): حيث يتم توثيق المتطلبات التي تم الاعتماد عليها في النهاية.

المرحلة الثالثة: Requirements documentation/specification

يتم في هذه المرحلة كتابة وثائق المتطلبات، إذ يجب تدوين المتطلبات الأساسية لتوثيق الاتفاق الذي تم بين الشركة المطورة والعميل ولتكون هذه الوثائق مرجعاً للمطورين developers أثناء عملية التطوير.

المرحلة الرابعة: Requirements Validation

هي مرحلة التأكد من صحة ما كُتب في الوثائق، والتأكد من مطابقته لما يريده الزبون.

المعايير التي يتم وفقها التحقق من صحة المتطلبات:

Validity
(الصلاحية)

Consistency
(التناسق)

Completeness
(الشمولية)

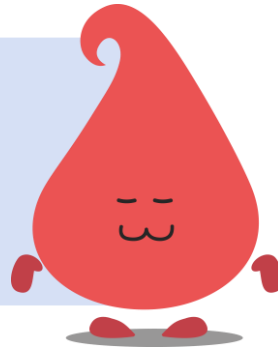
Realism
(الواقعية)

Verifiability
(القابلية للتحقق)

المرحلة الخامسة: Requirement Management

إدارة المتطلبات هي عملية إدارة المتطلبات المتغيرة خلال عملية هندسة المتطلبات وتطوير النظام، حيث من الممكن ظهور متطلبات جديدة أثناء التطوير وبعد بدء الاستخدام.

Success is a journey,
not a destination.



بعد أن تحدثنا عن أولى مراحل ال SDLC وهي التواصل Communication وجمع المتطلبات، سيتم التطرق الى الحديث عن المرحلة الثانية Planning فيما بعد، أما الآن فستتكم عن المرحلة الثالثة وهي النمذجة Modeling.

2- System modeling (نمذجة النظام):

هي عملية إنشاء نماذج (Models) تمثل النظام البرمجي بشكل مجرد، أي دون الدخول في تفاصيل التنفيذ البرمجي، حيث يقدم كل نموذج منظوراً (Perspective) مختلفاً للنظام، ويسلط الضوء على جانب محدد منه. بمعنى آخر، النمذجة لا تهدف إلى بناء النظام نفسه، وإنما إلى تمثيله وفهمه قبل بنائه.

الفائدة من النماذج:

1. تحسين التواصل مع أصحاب المصلحة (Stakeholders Communication)

2. تقليل التكلفة والمخاطر

تصنيف المخططات:

يمكننا تصنيف المخططات إلى ثلاث تصنيفات رئيسية تدرج تحتها تصنيفات فرعية، وهي:

- من حيث المنظور Perspective (سياقي - تفاعلي - هيكلي - سلوكي).
- من حيث الهدف من بنائه والمرحلة phase التي يتم بناؤه فيها (تحليلي - تصميمي).
- من حيث الطبقة layer التي يمثلها في النظام (interface-logic-Entity).

يوجد لكل مخطط وظيفة محددة وهدف محدد، لذلك وُجدت عدة أبعاد وكل نموذج أو مخطط يمكن أن يكون تابع لبُعد معين أو لأكثر من بُعد فيمكن أن تشكل كل نقطة التقاء بين الأبعاد الثلاثة نموذجاً لمخطط معين.

البعد الأول: منظورات النمذجة (System Modeling Perspectives/aspects):

تعتمد على أربعة منظورات رئيسية:

1. المنظور الخارجي أو السياقي (External/Contextual Perspective): تعبر عن سياق وبيئة النظام الخارجية.
2. المنظور التفاعلي (Interaction Perspective): تحدد كيفية وآليات التفاعل بين النظام وبيئته الخارجية
3. المنظور البنيوي (Structural Perspective): تركز على بنية النظام الداخلية
4. المنظور السلوكي (Behavioral Perspective): تركز على كيفية تفاعل مكونات النظام الداخلية بين بعضها البعض

البعد الثاني: المرحلة phase

يمكن تقسيم النماذج من حيث الهدف إلى:

1. النماذج التحليلية (Analysis Models)

تركز على فهم المشكلة وتحليل وتوصيف المتطلبات وتهتم بمعرفة من هم الـ actors وكيف يتفاعلون مع النظام وبالتالي تركز على الجزء الخارجي من النظام وعلى المخططات السياقية والتفاعلية

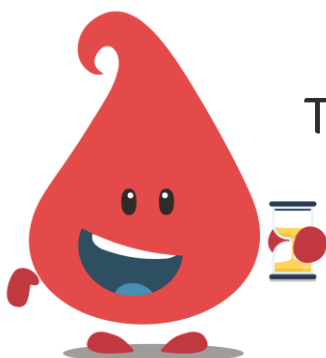
2. النماذج التصميمية (Design Models)

تركز على بناء الحل وبناء التطبيق وتمثل الشكل النهائي للنظام قبل التنفيذ البرمجي، تستخدم كمرجع أثناء عملية التطوير، فهي تهتم بالجزء الداخلي من النظام ومعرفة الـ components التي يتألف منها وكيف تتفاعل مع بعضها وبالتالي يركز على المخططات البنيوية والسلوكية.

البعد الثالث: طبقات النظام (System layers)

يمكن إسقاط النماذج على طبقات النظام المختلفة مثل:

1. طبقة العرض والواجهات (Presentation / interfaces Layer)
2. طبقة المحتوى والمنطق البرمجي (Content / Logic Layer)
3. طبقة الكيانات وقواعد البيانات (Data / Entity Layer)



The best time to start is now

”أفضل وقت للبدء هو الآن“

UML Diagrams

تكتب معظم نماذج البرمجيات باستخدام UML، وهي لغة نمذجة موحدة توفر معنى واضح للرموز المستخدمة في البرامج.

مخطط حالات الاستخدام (Use Case Diagram)

يستخدم هذا المخطط بشكل أساسي في مرحلة استخلاص المتطلبات (Requirements Elicitation) لتقديم نظرة عامة شاملة حول الخدمات التي يقدمها النظام ومستخدميه.

المكونات الأساسية:

1. حالة الاستخدام (Use Case): هي جزء فرعي (Subset) من الوظائف الكلية للنظام، وتُصنف إلى "Business Use Case" و "System Use Case" ترسم كشكل بيضوي أفقي.
2. الفاعل (Actor): أي كيان (إنسان، نظام آخر، أو حتى الزمن) يتبادل المعلومات مع النظام. ينقسم إلى:
 - ❖ Business Actor: المستفيد من الخدمة
 - ❖ System Actor: من يتفاعل بشكل مباشر مع النظام ، و ينقسم إلى نوعين:
 - ✓ (Primary): هو من يبدأ التفاعل لطلب الخدمة، يقوم بإجراء Initialization لخدمات النظام
 - ✓ (Secondary): ميسر للطلب (مثل البنك في عمليات الدفع). وقد يكون Server (يستجيب للطلب) أو Receiver (يستلم شيئاً من النظام دون رد).
3. حدود المنظومة (System Boundary): الإطار الذي يفصل داخل النظام عن بيئته الخارجية.

العلاقات في مخطط حالات الاستخدام:

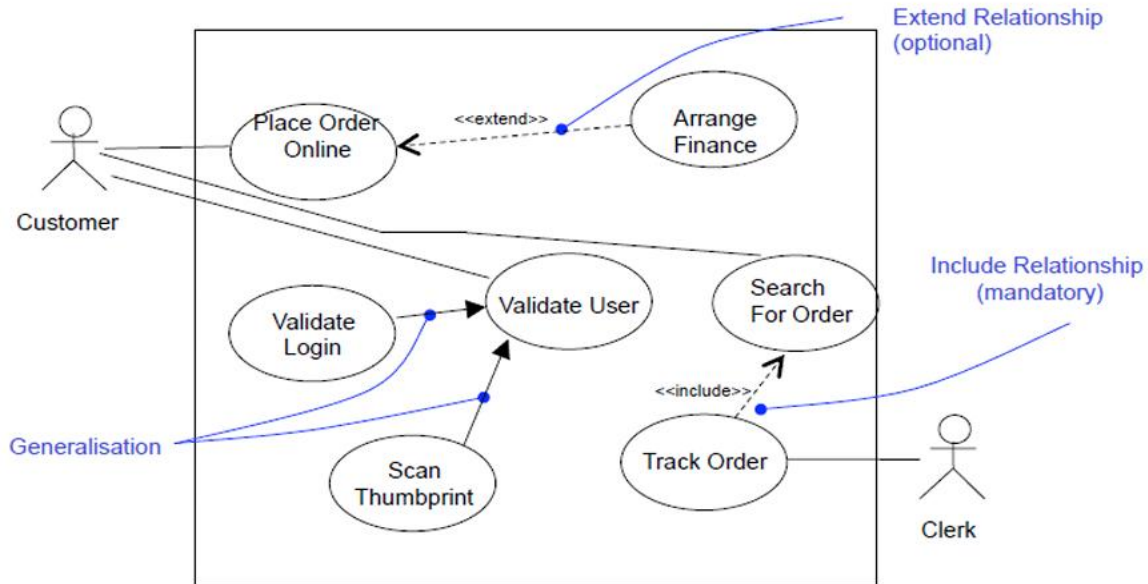
- (Association): يربط الفاعل بحالة الاستخدام ويحدد صلاحياته.
- (Include/Abstraction) علاقة إجبارية: تكون حالة الاستخدام "جزءاً لا يتجزأ" من حالة أخرى ولا تكتمل بدونه
- (Extend): علاقة اختيارية: حيث توسع حالة استخدام وظائف حالة أخرى تحت شرط أن تتم الحالة الأساسية أولاً
- (Dependency): تشير إلى أن تنفيذ حالة استخدام يعتمد على تحقق حالة أخرى مسبقاً.
- (Generalization / inheritance): تستخدم لتقليل التكرار، حيث يرث فاعل أو حالة استخدام خصائص وتفاعلات "الأب".

Template for detailed descriptions of use cases (Use Case Narrative)

بما أن المخطط البصري وحده لا يكفي، يتم إرفاقه بجدول تفصيلي يشمل:

- ID: عبارة عن primary key الخاص بالـ use case.
- Use case name: اسم حالة الاستخدام وهي جملة فعلية نشطة قصيرة تصف مهمة معينة.
- Actors
- Trigger: هي الحدث المحرض لبدء حالة الاستخدام.
- Pre-conditions: الشروط الأساسية التي يجب أن تتحقق قبل بدء الخدمة.

- Basic flow: وصف لمسار الأنشطة الذي يحدث عادةً لتنفيذ العملية المحددة في حالة الاستخدام.
- Alternate Flows: وصف للمسارات البديلة لتنفيذ العملية (مثلاً تفرعات معينة عند حدوث شرط ما).
- Exception Flow: توضح المسارات عند حدوث أخطاء محتمل حدوثها.
- Post-condition: مجموعة من الشروط التي يجب أن تتحقق بعد انتهاء العملية.



ملاحظة: Use Case Diagram هو لوحده مخطط سياقي ومع جداول ال use case يصبح تفاعلي أيضاً وهو مخطط تحليلي وليس تصميمي

مخطط الأنشطة (Activity Diagram)

يهدف إلى تمثيل تدفق الأنشطة (Flow of Activities) اللازمة لانجاز خدمة معينة ويعد أداة بصرية مكملية للجداول النصية لحالات الاستخدام.

■ الرموز الأساسية (Flow Objects):

- الأحداث (Events): تحدد نقطة البداية (Start) ونقطة النهاية (End).
- النشاط (Activity): خطوة عمل تكتب بصيغة (فعل + اسم).
- البوابات (Gateways):

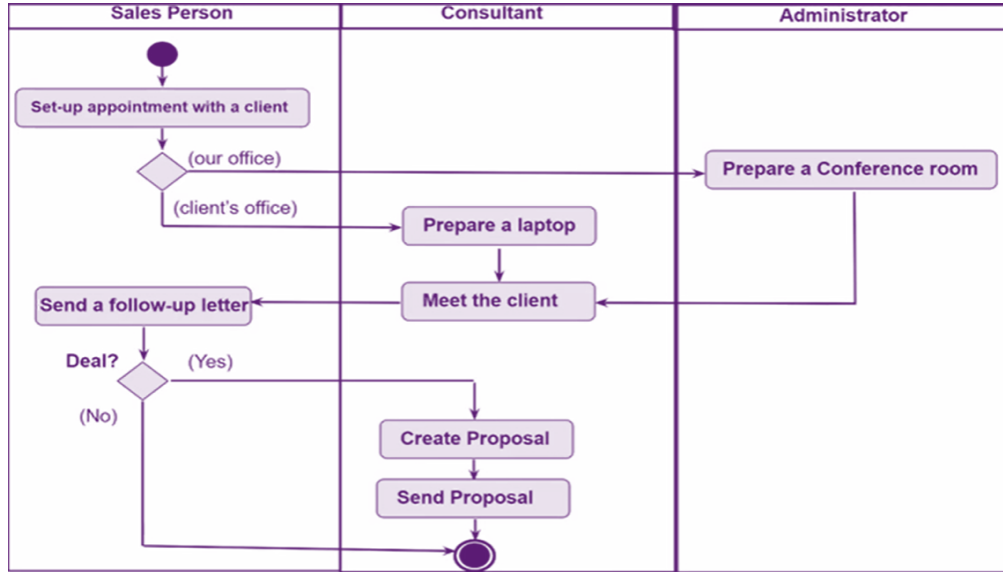
- ✓ XOR (Exclusive): تسمح بمرور مسار واحد فقط من عدة خيارات (حالات else-if).
- ✓ AND (Parallel): تسمح بتنفيذ عدة مسارات في آن واحد، ولا ينشط الخرج إلا باكتمالها جميعاً.

■ علاقات الربط (Connecting Objects):

- Sequence Flow: سهم مستمر يوضح تسلسل التنفيذ.
- Message Flow: سهم متقطع يمثل تبادل الرسائل بين مشاركين منفصلين (مثل الزبون والنظام).
- Association: سهم منقط يستخدم لربط الأنشطة بالكائنات الإضافية للمخطط مثل ملفات البيانات.

المسارات (Swimlanes): تستخدم لتنظيم المخطط وتحديد المسؤول عن كل نشاط عبر:

- الحوض (Pool): يمثل جهة كاملة (مثل النظام أو المستخدم).
- المسار (Lane): تقسيمات داخل الحوض لتوزيع المهام بدقة.



مخطط الأنصاف (Class Diagram)

يعد من أهم المخططات الهيكلية (Structural Models)، ويستخدم لتمثيل البنية الداخلية للنظام من خلال توضيح الأنصاف (Classes)، خصائصها (Attributes)، سلوكياتها (Methods)، والعلاقات بينها

المفاهيم الأساسية:

- الكائن (Object): أي شيء له بيانات وسلوك (شخص، مكان، حدث).
- الصف (Class): قالب عام لمجموعة كائنات تشترك في الخصائص والسلوك.
- الخصائص (Attributes): البيانات التي تمثل الخصائص المتعلقة بالكائن
- السلوك (Behavior): مجموعة الأفعال أو الوظائف التي يمكن للكائن تنفيذها
- التغليف (Encapsulation): تجميع البيانات والسلوك في وحدة واحدة (الصف) والتحكم بالوصول إليها.
- Object Instance: يمثل نسخة محددة من الكائن، تحتوي على قيم فعلية للخصائص المرتبطة به.

العلاقات في مخطط الأنصاف:

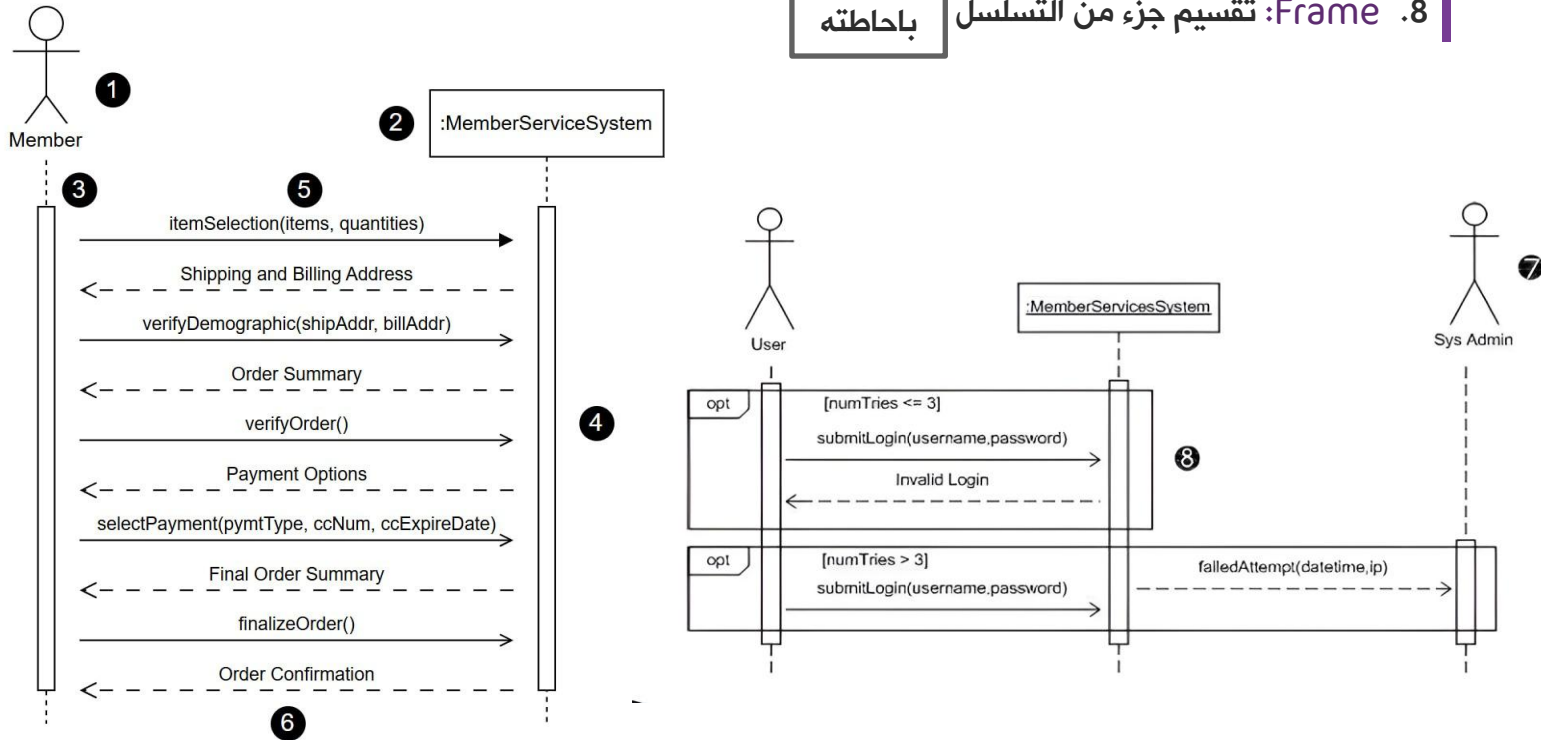
- الوراثة (Inheritance/Generalization): علاقة "is a" أو "is like a" حيث يرث الصف الفرعي (Subtype) السمات والسلوكيات من الصف الأعلى (Supertype)
- الارتباط (Association): علاقة عمل طبيعية وتفاعل بين صنفين أو أكثر، وتحدد التعددية (Multiplicity) عدد الكائنات المرتبطة (مثل 1..1 أو 1..*)
- علاقة الكل والجزء (Whole-Part Relationship):
 - ✓ (Composition): علاقة قوية؛ عمر الجزء مرتبط بالكل (إذا دُمِر الكل دُمِر الجزء).
 - ✓ (Aggregation): علاقة ضعيفة؛ يمكن للجزء البقاء مستقلاً بعد زوال الكل.

مخطط التسلسل (Sequence Diagram)

هو مخطط تفاعلي وسلوكي يوضح تسلسل التفاعل زمنياً بين الكائنات والممثل (Actor) لتحقيق حالة استخدام معينة.

المكونات الأساسية:

1. Actor
2. System: يشير الصندوق إلى النظام كنظام متكامل
3. خط الحياة (Lifeline): يمثل الفترة الزمنية لوجود الكائن في الذاكرة.
4. مستطيل التفعيل (Activation Bar): يمثل الفترة التي يكون فيها الكائن نشطاً ينفذ عمليات
5. رسائل الإدخال: تكون من Actor إلى System.
6. رسائل الإخراج: تكون من System إلى Actor (تكون بخط متقطع).
7. Actor Receiver: الفاعل أو النظام الخارجي الذي يستقبل الرسائل.
8. Frame: تقسيم جزء من التسلسل **باحتاطه**



ملاحظة: وفقاً لقواعد الرسم الأكاديمية، لا يتم تضمين قاعدة البيانات (Database) ككائن في مخطط التسلسل لأنها تُعتبر نظاماً خارجياً مستقلاً أو وسيلة تخزين.

Draw your future

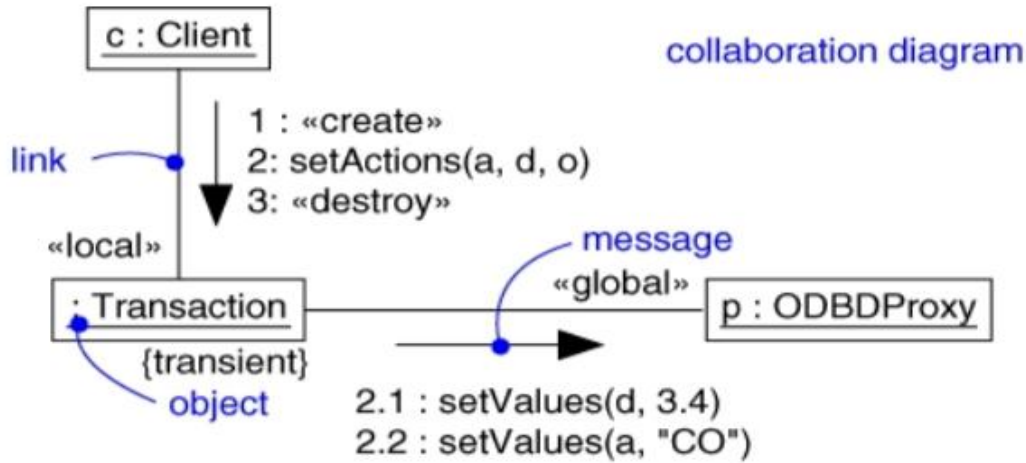


مخطط (Collaboration/Communication Diagram)

هو مخطط تفاعلي وسلوكي يوضح نفس معلومات مخطط التسلسل ولكن بتركيز مختلف.

التركيز: يركز على البنية والعلاقات بين الكائنات بدلاً من المحور الزمني.

الترتيب: يُحدد تسلسل التفاعلات فيه عن طريق ترقيم التفاعلات (1, 1.1, 1.2).

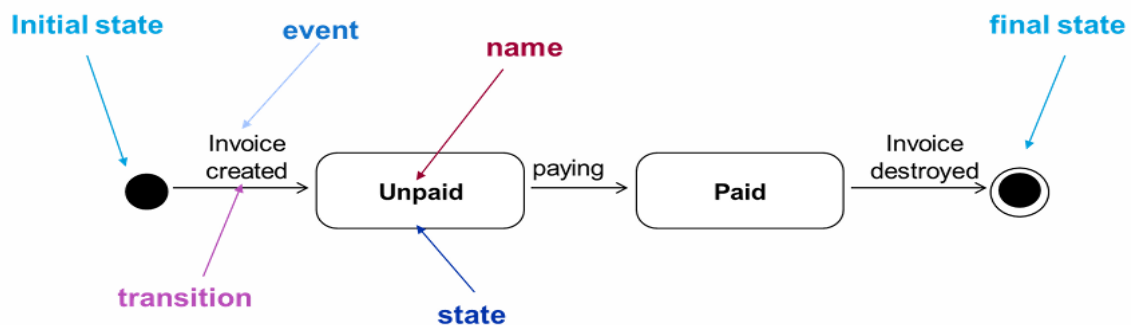


مخطط الحالة (State Diagram)

على عكس مخطط التسلسل الذي يركز على عدة كائنات، فإن مخطط الحالة يعمل كعملية "Zoom-in" على كائن واحد فقط لوصف سلوكه خلال دورة حياته.

مكوناته:

- الحالة (State): الوضع الذي يكون فيه الكائن في لحظة معينة (مثلاً: Unpaid, Paid).
- الحدث (Event): المسبب الذي يؤدي لتغيير الحالة (إشارة، وقت، أو تحقق شرط).
- الانتقال (Transition): العلاقة التي تربط بين حالتين، وتشير إلى انتقال الكائن من الحالة المصدرية إلى الحالة الهدف.
- Action: إجراء يحدث نتيجة الانتقال من حالة إلى أخرى.



System Design

تعدّ مرحلة التصميم (Design) من أهم مراحل دورة حياة تطوير البرمجيات (SDLC) ، إذ تمثل الجسر الذي يربط بين ما تم تحديده في مرحلة التحليل (Analysis) وبين مرحلة التنفيذ (Implementation).

نماذج التحليل ونماذج التصميم:

- نماذج التحليل (Analysis Models): تمثل ما يحتاج النظام إلى فعله (WHAT).
- نماذج التصميم (Design Models): فتمثل كيف سينفذ النظام هذه المتطلبات (HOW).

■ تصميم النظام (System Design)

يهدف تصميم النظام إلى:

- وصف وتنظيم وهيكلية جميع مكونات النظام.
- تحديد معمارية النظام والكائنات التفصيلية.
- تمكين بناء النظام ونشره (Deployment).

ينقسم تصميم النظام إلى مستويين:

1. التصميم المعماري عالي المستوى (Architectural Design) :
يهتم بالبنية التحتية العامة مثل العتاد (Hardware) ، الشبكة (Network) ، وبرمجيات النظام.
2. التصميم التفصيلي (Detail Design) :
يركّز على الوحدات الصغيرة وتصميم البرمجيات التفصيلي لحالات استخدام محددة.

■ Architecture in the Small and in the Large

- **Architecture in the Small**: تهتم بهيكلية البرامج الفردية، وكيفية تفكيك برنامج واحد إلى مكونات مترابطة.
- **Architecture in the Large**: تهتم بهيكلية الأنظمة المؤسسية المعقدة التي تتكون من عدة أنظمة فرعية موزعة على حواسيب مختلفة وقد تكون مملوكة لجهات مختلفة

الأنماط المعمارية (Architectural Patterns)

1. Layered Architecture

تنظم النظام إلى طبقات، حيث تقدم كل طبقة خدمات للطبقة التي فوقها.

2. Client-Server Architecture

تنقسم النظام إلى خدمات حيث يتم تقديم كل خدمة من خادم Server وهناك clients يطلبون الخدمات.

3. MVC(Model-View Controller)

يفصل بين البيانات (Model) ، العرض (View) ، والتفاعل (Controller)

4. Repository Pattern

تدار البيانات في مستودع مركزي تتفاعل معه جميع المكونات.

5. Pipe and Filter

تنظيم المعالجة كسلسلة من التحويلات المتتالية.

Detail System Design

تحتوي الأنشطة عالية المستوى (Higher-level Activities) على العديد من الأنشطة منخفضة المستوى (Lower-level Activities) وتتفاعل معها، تشمل هذه الأنشطة:

1. تصميم خدمات الدعم (Design Support Services)
2. تصميم هيكلية البرمجيات (Design Software Architecture)
3. تصميم حالة الاستخدام (Design the Use Case Realization)
4. تصميم قاعدة البيانات (Design the Database)
5. تصميم واجهات النظام والمستخدم (Design the System and User Interfaces)
6. تصميم الأمن والضوابط (Design Security and Controls)

Design Models

إن النماذج الأساسية (Primary Models) التي يتم تطويرها أثناء التصميم هي:

- مخطط فئات التصميم (Design Class Diagram)
- مخططات التسلسل (Sequence Diagrams)
- مخططات الحالة (State-Machine Diagrams)

بعض مبادئ Design Class الأساسية:

■ الاقتران (Coupling)

يشير إلى مدى اعتماد الفئات (Classes) على بعضها البعض وإلى درجة الترابط بين مكونات النظام البرمجي، أي يقيس الاقتران مقدار المعرفة التي تمتلكها فئة ما عن فئة أخرى، ومدى حاجتها لها لكي تعمل بشكل صحيح. كلما زاد الاقتران بين الفئات، أصبح النظام أصعب في التعديل والصيانة، فعندما يكون الاقتران عالياً تكون الفئات

مرتبطة بشكل مباشر وقوي وأي تعديل بسيط في class واحد قد يُجبرنا على تعديل عدة classes أخرى وإعادة اختبار أجزاء كبيرة من النظام لذلك تعد تكلفته عالية.

■ التماسك (Cohesion)

يقيس مدى تناسق وترابط الوظائف داخل ال class الواحد، فيكون ال class متماسك يحوي على جميع العمليات التي يحتاجها بداخله وقادر على تلبية نفسه بنفسه دون الاحتياج الى الارتباط مع باقي ال Classes والتخاطب معها لإنجاز مهامه، فيكون ال class قادراً على أداء مسؤولية واحدة واضحة بمفرده.

■ فصل المسؤوليات (Separation of Responsibilities)

تقسيم class إلى عدة Classes ذات تماسك عال (Highly Cohesive Classes)

Tiers & Layers

Layers (الطبقات المنطقية)

تمثل فصلاً منطقياً داخل التطبيق نفسه، مثل:

- View Layer
- Business Logic Layer
- Data Access Layer

قد تكون جميع الطبقات موجودة على نفس الجهاز.

Tiers (الطبقات الفيزيائية)

تمثل فصلاً فيزيائياً على أجهزة مختلفة، مثل:

- Client Tier
- Application Server Tier
- Database Server Tier

SOLID Principles

تمثل SOLID مجموعة من المبادئ الأساسية لتصميم البرمجيات القائمة على البرمجة غرضية التوجه حيث تساعد هذه المبادئ على تحسين قابلية الصيانة، والمرونة، وإعادة الاستخدام.

1. Single Responsibility Principle (SRP)

ينص هذا المبدأ على أن لكل كلاس سبب واحد للتغيير.

2. Open / Closed Principle (OCP)

يجب أن تكون كيانات البرمجيات (classes, modules, functions, etc..) مفتوحة للتمديد ولكن مغلقة للتعديل.

3. Liskov Substitution Principle (LSP)

يجب أن يستطيع الكلاس الابن أن يحل محل الأب دون كسر سلوك النظام.

4. Interface Segregation Principle (ISP)

مبدأ فصل الواجهات.

5. Dependency Inversion Principle (DIP)

لا يجب أن تعتمد الوحدات عالية المستوى على الوحدات منخفضة المستوى.

Microservices Architecture

تعد Microservices Architecture أحد الأساليب المعمارية الحديثة في تصميم الأنظمة البرمجية، وتهدف إلى بناء النظام على شكل مجموعة من الوحدات Modules أو الخدمات الصغيرة.

في معمارية Microservices :

يتم تقسيم النظام إلى خدمات مستقلة (Services) كل خدمة:

- تطوّر بشكل مستقل
- تنشر (Deploy) بشكل مستقل
- تدار وتحدّث دون التأثير المباشر على باقي الخدمات

تتميز معمارية Microservices بعدة خصائص أساسية:

- الاستقلالية (Independence)
- ضعف الاقتران (Loose Coupling)
- التواصل عبر الشبكة (Communication over Network)
- قابلية التوسع (Scalability)
- تعدد التقنيات (Technology Diversity)

هذا النمط يدعم تحقيق المتطلبات غير الوظيفية مثل:

- Scalability
- Reliability
- Availability
- Maintainability

Umbrella Activities

هي مجموعة من الخطوات والإجراءات التي يتبعها فريق هندسة البرمجيات للحفاظ على التقدم والجودة وإدارة التغيير والمخاطر أثناء عملية التطوير، ليس الهدف منها الوصول إلى منتج، وإنما هدفها الحماية والدعم، وتعدّ ضمان لعمل ال Process Model بشكل صحيح.

أمثلة عن ال Umbrella activities:

SW metrics (مؤشرات أداء البرنامج):
قياس جودة المنتج البرمجي

Project management
(إدارة المشاريع)

:Pricing & Scheduling
تحديد زمن وكلفة إنجاز المشروع.

Quality management
(إدارة الجودة)

Risk management
(إدارة المخاطر)

وهذه الأنشطة ستكون محور مادتنا لهذا الفصل.

لكن لن نتوسع كثيراً في (Project Management)، لأن هناك مادة مستقلة في السنة الخامسة تتناول إدارة المشاريع، وستجدون فيها تداخلاً (Overlapping) مع ما ندرسه، لكننا هنا نركّز بشكل خاص على إدارة مشاريع البرمجيات (Software Project Management).

من أهم النقاط في هذا المجال أن نفهم إيجابيات إدارة المشروع وعوامل النجاح (Success Factors)، لأن الهدف من إدارة المشاريع ليس الفشل، بل تحقيق النجاح. لذلك، يجب أن نحدد معايير نجاح المشروع.

أهم هذه المعايير:

- إذا لم يكن لديك تخطيط (Planning)، فلن تكون لديك توقعات واضحة (Expectations).
- إذا لم تحدد الأساسيات والموارد والأنشطة ونقطة النهاية، فلن تستطيع معرفة ما إذا كنت قد وصلت إلى الهدف أم لا.

بالتالي، النجاح مرتبط بشكل أساسي بوجود التخطيط.

ضمن التخطيط، نأخذ بعين الاعتبار الجدولة (Scheduling)، حيث يجب أن نعرف:

- ✓ ماذا سنفعل؟
- ✓ متى سنفعله؟
- ✓ وكم سيستغرق من الوقت؟

ولكي تتمكن من تقدير الوقت، يجب تقسيم الأهداف إلى مجموعة من المهام (Tasks)، ومن ثم تخصيص الموارد (Resources) المناسبة لكل مهمة، بحيث تكون قابلة للتنفيذ ضمن الواقع.

مثال توضيحي:

إذا كان هدفك حضور محاضرة هندسة البرمجيات، فقم بتخطيط المنبه وتحضير نفسك، لكن واجهتك مشاكل مثل عدم توفر وسيلة نقل أو عدم وجود مال، أو انقطاع الطريق، فتأخرت عن المحاضرة ولم يُسمح لك بالدخول.

في الواقع، هذه الحالة تسمى إدارة المخاطر (Risk Management).

فهل يجب دراسة المخاطر في كل المشاريع؟

في المشاريع الصغيرة، غالباً ما تتم هذه العملية بشكل غير رسمي (ضمن التفكير الداخلي). أما في المشاريع الكبيرة، فيجب توثيقها بشكل واضح منذ البداية، لأن تجاهلها يؤدي إلى مشاكل لاحقاً.

قبل إدارة المخاطر، هناك نقطة مهمة:

عندما تخطط، فأنت تتحدث عن المستقبل. وعندما يصبح المستقبل حاضراً، كيف تعرف إن كنت قد نجحت أم لا؟ فأنت أمام خياران:

- متابعة تنفيذ المهام (Tasks)
- أو متابعة النتائج (Outputs)

في كلتا الحالتين، تحتاج إلى بيانات دقيقة لأن الاعتماد على آراء الأشخاص (Opinions) غير كافٍ، وقد يؤدي إلى مشاكل، لذلك يجب الاعتماد على الحقائق (Facts).

والحقائق تعني وجود:

- مقاييس واضحة
- أدوات قياس
- بيانات قابلة للتحقق

ازرع جميلاً ولو في غير موضعه، فلن
يضيع جميل أينما زرع



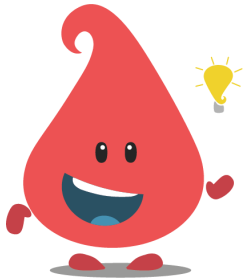
- Software Metrics ليست مجرد بيانات، بل هي أساليب وطرق لقياس خصائص البرمجيات بشكل كمي (Quantitative Measurement) مثل الجودة والأداء وعدد الأخطاء وزمن التنفيذ، بهدف تحويل “الملاحظات” إلى أرقام قابلة للتحليل واتخاذ القرار.
- بعد ذلك ننتقل إلى إدارة الجودة (Quality Management)، وهي مجال مهم جدًا، ويُعد من التخصصات المطلوبة، خاصة في الدول التي تتجه نحو الانفتاح.

هل ضمان الجودة (Quality Assurance) هو نفسه عمل المختبر (Tester)؟

Quality Assurance (QA) ليست اختبارًا فقط، بل هي عملية أوسع تركز على تحسين العمليات ومنع الأخطاء قبل حدوثها، بينما الاختبار (Testing / Quality Control) يركز على اكتشاف الأخطاء في المنتج بعد أو أثناء تطويره.

في بعض نماذج العمليات (Process Models)، يُعتبر الاختبار (Testing) مرحلة مستقلة (Milestone)، ولكن في نموذج دورة حياة تطوير البرمجيات (SDLC) الذي نناقشه حاليًا، لا يُعد الاختبار مرحلة منفصلة، وذلك لأنه نشاط موجود في أكثر من مرحلة، ويستمر بشكل متكرر خلال التطوير وليس فقط ضمن مرحلة واحدة.

سؤال متقدم:



برأيكم، ما النشاط (Activity) المشترك بين مرحلتي
Construction و Deployment؟
وما المقصود بمرحلة Construction؟

والان بداية موفقة مع مادة هندسة البرمجيات 2

